PROMIN STANDARD

promin

version 1.0.0-alpha.4

Contents

Standard

Canonical standard version: 1.0.0-alpha.4. promin defines a minimal, verifiable standard for agent work in which semantics, authority, evidence, and events have unambiguous owners. JSON Core is normative; this text is a generated operational projection of the same facts.

Owner: core/promin.manifest.json | Validator: bundle_digest + core_components

Canonical owners

Core file	Sole responsibility
core/promin.manifest.json	Own Core bundle identity and exact digest binding for the five other Core artifacts.
core/semantic-model.json	Own the minimal persistent vocabulary, relation grammar, derived-projection boundary, and provider capability contracts.
core/authority-model.json	Own action capabilities, runtime Grant semantics, trust modes, configured signature-adapter verification, root ceilings, and separation-of-duty constraints.
core/policy-set.json	Own executable cross-record invariants through stable validator identifiers and no duplicated budget values.
core/conformance.json	Own acceptance predicates, mutation families, structural hard ceilings, scale contracts, and production-claim rules.
core/contracts.schema.json	Own all structural, format, kind-specific payload, and strict critical-field contracts in one bundle.

The selected preset is outside Core identity. It chooses one bounded operating profile and provider needs but grants no authority. The schema is a deterministic Draft 2020-12 projection compiled from the canonical owners, never a second semantic or policy owner.

Owner: core/promin.manifest.json; presets/semantic-standard.json | Validator: exact six-file Core + selected preset outside Core

Package contents

The canonical tree contains exactly 187 regular files under 16 declared directories. MANIFEST.json enumerates 185 payload files; MANIFEST.json, SHA256SUMS.txt are the generated integrity surfaces. Missing, additional, linked, special, or transient paths reject.

Location	Regular files
package root	14
.github/	1
capability_profiles/	2
core/	6
docs/	19
examples/	1
human/	4
language_profiles/	1
presets/	1
profiles/	12
promin/	51
prompts/	2
skills/	4
tests/	56
tools/	13

Root files are exactly .gitattributes, .gitignore, CONTRIBUTING.md, LICENSE, MACHINE_README.md, MANIFEST.json, NOTICE, README.md, SECURITY.md, SHA256SUMS.txt, THIRD_PARTY_NOTICES.md, TRADEMARKS.md, VERSION.json, pyproject.toml. License, governance, dependency, and third-party notice closure is part of package identity.

Owner: MANIFEST.json; SHA256SUMS.txt; tools/promin_validate.py | Validator: CANONICAL_PACKAGE_FILES + CANONICAL_PACKAGE_DIRECTORIES

Purpose and boundaries

The standard separates normative state from derived indexes and reports, authority from roles and leases, and execution results from acceptance decisions. Model tier may change analysis depth, decomposition, and parallelism, but never authority.

- promin is not a distributed-consensus system.
- A projection, search rank, report, or preset is neither normative authority nor an authority grant.
- A Lease is not acceptance; execution, validation, promotion, and final decision are distinct actions.
- Reference-package success is insufficient for production credit without P0/P1 closure, rollback/recutover parity, and a duly authorized Decision.

Owner: core/semantic-model.json; core/authority-model.json; core/conformance.json | Validator: model_tier_rule + production_claim_rule

Semantic model and granularity

Semantic, public, compilation, runtime-dispatch, and documentation granularities are independent. A separate type or object is justified only by an independent invariant, policy, replaceability, measurable algorithm, failure boundary, reuse, or domain significance.

Owner: core/semantic-model.json | Validator: design_rules

Persistent entities

Kind	Purpose
Task	Represent one schedulable unit with one acceptance predicate and bounded mutation scope.
Candidate	Identify either one provider-proven immutable product snapshot or one explicitly non-creditable observation, independently from control state and bound to the exact candidate recipe.
Artifact	Identify immutable product, evidence, log, report, or diff bytes with provenance and retention metadata; raw-file inventory produces only derived Artifact proxies, while a diff may carry the canonical Candidate-delta change manifest without creating another persistent entity.
Run	Record one execution or validation attempt against exact candidate, policy, tool, and input digests.
Finding	Record an observed defect or policy violation with explicit effective-blocking semantics.
Decision	Record a project-operational acceptance, rejection, resolution, waiver, promotion, or release judgement. It is distinct from the external StandardReleaseDecision for the standard distribution.
GateResult	Record one gate outcome against the exact immutable Task-owned GateRunDefinition, current Run, and finalized evidence Artifact records; skipped, blocked, failed, stale, or unresolved evidence never receives pass credit.
Grant	Prove one subject capability within one activation, time interval, nonce, and scope.
Lease	Coordinate temporary mutation ownership through generation, fencing, expiry, heartbeat, and close acknowledgement.

Owner: core/semantic-model.json | Validator: admission.entity

Relations

Kind	Source	Target	Rules
DEPENDS_ON	Task	Task	acyclic-for-ready-frontier, accepted-current-outcome-required
READS	Task, Run	Candidate, Artifact	read-does-not-imply-authority
PRODUCES	Task, Run	Candidate, Artifact, Finding, Decision	content-or-record-identity-required
VALIDATES	Run	Candidate, Artifact, Task	candidate-policy-tool-input-binding-required
CITES	Finding, Decision	Artifact, Run, Finding	selector-and-content-resolution-required
BLOCKS	Finding	Task, Candidate, Decision	effective-only-while-open-and-blocking

Owner: core/semantic-model.json | Validator: admission.relation

Derived boundary

- InventoryEntry

- SearchDocument
- InventoryProjectionRow
- ProjectionLimits
- RetrievalPage
- ReadyFrontier
- WorkCardProjection
- WorkCard
- NextResult
- ContinueResult
- NavigationView
- StateCheckpoint
- CurrentReleaseStatus
- DoctorResult
- OperationMetrics
- Report

Each raw inventory file yields exactly one derived Artifact proxy. Raw-file proxies never create a persistent Task or relation; persistence still requires an independent invariant.

Owner: core/semantic-model.json | Validator: derived_projection_kinds

Inventory

Inventory reads the product tree once and writes an immutable manifest-verified JSONL stream. Each row is normalized and folded into rolling identity and stream digests, byte and row counts; publication uses atomic replacement. Projection rebuild verifies that descriptor while consuming the stream once and does not materialize the corpus in memory. One accepted raw file produces exactly one derived Artifact proxy with data_class=untrusted-source. Inventory never synthesizes Task, READS, or PRODUCES facts.

InventoryProjectionRow	Structural rule
digest	$ref="#/$defs/Digest"
path	$ref="#/$defs/RelativePath"
record_type	const="InventoryProjectionRow"
search_text	type="string"; minLength=1; maxLength=4096
semantic_proxy	type="object"; additionalProperties=false
size	type="integer"; minimum=0

Stream invariant	Value
manifest_verified_jsonl	True
project_tree_passes_max	1
memory_amplification_max	32
derived_artifact_proxies_per_raw_file	1
search_text_bytes_max	4096

- P-008 minimal-graph: Each verified raw file creates exactly one non-authoritative derived Artifact proxy. Raw inventory creates no Task, READS, or PRODUCES record; Tasks and typed relations require explicit semantic content.
- P-017 single-scan-inventory: One requested inventory performs at most one full product-tree pass; all projections derive from its stream.

- P-043 candidate-snapshot-consistency: The candidate recipe is applied in the single inventory pass; promin v1 permits only provider-proven immutable VCS trees for credit, while best-effort observations are explicit and receive no pass credit.
- P-051 verified-inventory-provenance: Public rebuild accepts only a verified InventoryResult or its persisted manifest; caller provenance fields are reserved, data_class is forced to untrusted-source, and runtime injects the exact path and digest.

Owner: core/semantic-model.json; core/policy-set.json; core/contracts.schema.json | Validator: InventoryProjectionRow + manifest-verified JSONL + single_scan_inventory + inventory_projection_minimality

Initialization

`promin init` accepts an explicit bundle, preset, and exactly five plan files: project.json, standards.json, technologies.json, licenses.json, and authority.json. Team-signed trust additionally requires an explicit JSON array through --activation-proofs; local-owner derives its one root proof and rejects that option. Init verifies Core, preset, provider health, and licenses before commit; installs an immutable digest-addressed standard; and atomically writes exactly five init records: ProjectInit, StandardsInit, TechnologiesInit, AuthorityInit, and Activation. licenses.json is a required validation input, not a sixth installed record. Successful provider observations are digest-bound in a transient in-memory preflight receipt; that receipt is not an init record and is never written under `.promin/init/`. `--emit-plan` validates and canonicalizes the supplied plans outside `.promin/`, does not synthesize them from a project ID or source path, and rejects --activation-proofs; `--review-plan` validates inputs without writes; `--dry-run` adds provider preflight without mutation. Every init mode performs zero product-tree passes. The product repository needs no root core/; authority is only the exact immutable copy under `.promin/standard/<digest>/`.

Installed record	Schema	Required fields
project.json	ProjectInit	record_type, project_id, roots, candidate_recipe, preset_id, operating_profile, intent, resolved_profile, orchestration, telemetry
standards.json	StandardsInit	record_type, bindings
technologies.json	TechnologiesInit	record_type, bindings
authority.json	AuthorityInit	record_type, trust_mode, subjects, roots
activation.json	Activation	record_type, canonical_name, core_bundle_digest, preset_digest, init_file_digests, operating_profile, trust_mode, activation_digest, proofs, implementation_closure_digest

Owner: core/contracts.schema.json; core/policy-set.json | Validator: ProjectInit/StandardsInit/TechnologiesInit/AuthorityInit/Activation + exact_init_keyset

```
.promin/
  standard/<core-bundle-digest>/
    core/
    presets/<preset-digest>.json
  init/
    project.json
    standards.json
    technologies.json
    authority.json
    activation.json
  state/
    events/batches/
    objects/sha256/
    projection/
    head.json
    locks/
  generated/
  cache/
```

- P-002 activation-binding: Every authoritative record binds the exact active Core, preset, init, and profile configuration.
- P-023 exact-init-keyset: Activation binds exactly project.json, standards.json, technologies.json, and authority.json.
- P-028 exact-standard-files: Core and selected preset inputs reject symlinks and unexpected shadow files.

- P-030 configuration-exact-idempotency: Repeated initialization succeeds only when the requested canonical configuration digest exactly equals the existing Activation; otherwise it reports conflict.
- P-037 provider-preflight-before-activation: Every required provider identity and healthcheck succeeds before initialization is committed; a declared provider is not evidence of availability.

Owner: core/policy-set.json | Validator: activation_binding + exact_init_keyset + exact_standard_files + config_exact_idempotency + provider_preflight

Implementation closure

Each technologies binding names the exact promin runtime, Python interpreter, jsonschema, SQLite, and adapter components used by that capability. Activation and dependent Evidence and projection state bind the aggregate implementation_closure_digest. The runtime recomputes it before use; drift invalidates dependent current state without rewriting history.

Binding	Required content
TechnologiesInit.bindings[*].implementation_closure	runtime, interpreter, components, adapters, closure_digest
Activation.implementation_closure_digest	aggregate current implementation identity
EvidenceBinding.implementation_closure_digest	implementation used to produce evidence

- P-022 provider-substitution-binding: A provider identity or implementation-closure change invalidates the Activation and every dependent continuation, evidence Artifact, projection, replay result, and current release closure; it never silently changes authority.
- P-044 provider-adapter-dispatch: Every technologies.json binding resolves to the adapter actually invoked and carries a recomputed capability-specific implementation closure for the promin runtime, Python interpreter, jsonschema or SQLite component where used, and adapter. Unsupported or drifted tuples reject, and dependent evidence binds the exact implementation_closure_digest.

Owner: core/contracts.schema.json; core/policy-set.json | Validator: $defs/TechnologiesInit + $defs/Activation + $defs/EvidenceBinding + implementation_closure_binding

Authority and control

authority.json configures root ceilings and Activation only. Root command proofs bind one command intent and may issue only the first authority.manage Grant. Every Grant proof binds claim_digest; later Grants cite an active authority.manage issuer Grant or team signatures verified by the exact signature adapter selected in technologies.json.

Model strength changes decomposition, context, parallelism, and analysis depth; it never changes authority.

Default is deny. Maximum delegation depth is 8. Every Grant binds subject, capability, conjunctive effect scope, nonce, not-before and expiry, revocation state, exact child signed claim, issuer claim or configured team-signature boundary, and the exact Activation. Revocation Decisions and Events are resolved as immutable records; an arbitrary Decision ID is not proof.

Owner: core/authority-model.json | Validator: bootstrap_rule + default + model_tier_rule

Capabilities

Capability	Purpose
standard.activate	Activate one digest-bound repository configuration.
authority.manage	Issue or revoke runtime Grants within a configured root ceiling.
task.plan	Create, split, order, or block Tasks.
task.execute	Execute one Task inside its WorkCard and valid Lease.
lease.manage	Acquire, heartbeat, close, expire, or revoke Leases.
evidence.publish	Publish candidate-bound evidence Artifacts.
validation.evaluate	Evaluate gates without promotion or release authority.
finding.record	Record Findings.
finding.resolve	Resolve one Finding only with evidence bound to that Finding's canonical digest.
finding.waive	Waive one Finding only through an expiring exception-policy Decision, Finding-bound evidence, and separation of duties.
candidate.promote	Promote a validated Candidate.
release.decide	Record an immutable historical release judgement bound to an exact acceptance-closure and provider-binding digest; current eligibility remains derived.
export.create	Create a sanitized external package.
migration.manage	Execute cutover, rollback, and deterministic recutover.
projection.rebuild	Rebuild or verify non-authoritative projections.
projection.read	Read one bounded current projection or resume its continuation under the exact current Grant claim; it grants no mutation, validation, promotion, product acceptance, or standard distribution authority.

Owner: core/authority-model.json | Validator: capabilities

Separation of duties

ID	Rule	Constraint
SOD-001	lease-is-not-acceptance	{"forbid_lease_derived_capabilities": ["validation.evaluate", " finding.resolve", "finding.waive", "candidate.promote", "release.decide"], " scope_kind": "lease"}
SOD-002	executor-not-release-decider	{"forbid_same_subject_for_same_candidate": ["task.execute", " release.decide"], "scope_kind": "candidate"}
SOD-003	validator-not-self-promoter	{"forbid_same_subject_for_same_candidate": ["validation.evaluate", " candidate.promote"], "scope_kind": "candidate"}
SOD-004	finder-not-sole-waiver	{"forbid_same_subject_for_same_finding": ["finding.record", " finding.waive"], "scope_kind": "finding"}
SOD-005	evidence-producer-not-distribution-decider	{"forbid_same_key_capabilities": ["evidence.produce", " standard.distribute"], "forbid_same_subject_for_same_candidate": ["evidence.produce", "standard.distribute"], "scope_kind": " candidate-and-key"}

Standard distribution success never grants product acceptance. Live product credit separately requires a creditable immutable-vcs-tree Candidate, all P0/P1 findings closed with exact target-bound evidence, current project release closure eligibility, rollback and recutover equivalence, and an authorized human project Decision. product_acceptance_pass remains false for promin v1 distribution evidence.

Owner: core/authority-model.json; core/conformance.json | Validator: SOD-001..SOD-005 + signed standard-distribution decision

Tasks, leases, evidence, and decisions

Task and Lease change state only through policy-owned state machines. A mutation WorkCard requires a current Lease; a read-only WorkCard must not receive one. A Lease separately binds manager and holder Grants, generation, monotonic fence, heartbeat, expiry, and close acknowledgement, but never grants acceptance credit. ACTIVE and CLOSING, and EXPIRED or REVOKED without recorded reconciliation, continue to occupy capacity.

Task	Allowed next state
BLOCKED	READY, CANCELLED
CANCELLED	-

Task	Allowed next state
COMPLETED	-
LEASED	RUNNING, READY, BLOCKED, CANCELLED
PLANNED	READY, BLOCKED, CANCELLED
READY	LEASED, BLOCKED, CANCELLED
RUNNING	COMPLETED, BLOCKED, CANCELLED
Lease	**Allowed next state**
ACTIVE	CLOSING, EXPIRED, REVOKED
CLOSED	-
CLOSING	CLOSED, EXPIRED, REVOKED
EXPIRED	-
REVOKED	-

{'closed_release': {'requires_field': 'close_ack', 'slot_reusable': True, 'state': 'CLOSED'}, ' terminal_reconciliation': {'fencing_token_binding': 'exact-lease-fencing-token', 'field': ' capacity_reconciliation', 'generation_binding': 'exact-lease-generation', 'manager_capability': ' lease.manage', 'required_fields': ['reconciled_by', 'grant_id', 'grant_claim_digest', 'reconciled_at', ' generation', 'fencing_token'], 'slot_reusable_after_record': True, 'states': ['EXPIRED', 'REVOKED']}, ' unreconciled_slot_reusable': False}

Lifecycle command matrix

Command	Capability resolution	Effect scope
task.record	task.plan	{"command_kind": "task.record", "id_field": "task_id", " kind": "task", "mode": "payload-id"}
task.transition	task.plan	{"command_kind": "task.transition", "id_field": " task_id", "kind": "task", "mode": "payload-id"}
artifact.record	{"diff": "task.execute", "evidence": " evidence.publish", "log": " task.execute", "product": " task.execute", "report": " projection.rebuild"}	{"command_kind": "artifact.record", "id_field": " artifact_id", "kind": "artifact", "mode": "payload-id"}
run.record	{"execution": "task.execute", " validation": "validation.evaluate"}	{"command_kind": "run.record", "mode": " payload-multi-id", "targets": [{"id_field": "task_id", " kind": "task"}, {"id_field": "candidate_digest", "kind": " candidate"}]}
gate.record	validation.evaluate	{"command_kind": "gate.record", "id_field": " candidate_digest", "kind": "candidate", "mode": " payload-id"}
finding.record	finding.record	{"command_kind": "finding.record", "id_field": " finding_id", "kind": "finding", "mode": "payload-id"}
decision.record	{"accept": "validation.evaluate", " promote": "candidate.promote", " reject": "validation.evaluate", "release": " release.decide", "resolve": " finding.resolve", "revoke": " authority.manage", "waive": " finding.waive"}	{"command_kind": "decision.record", " grant_target_type": "Grant", "id_field": "target_id", " kind_field": "target_type", "kind_map": {"Candidate": " candidate", "Finding": "finding", "Task": "task"}, " mode": "payload-dynamic-id-or-referenced-grant-scope" }
lease.record	lease.manage	{"command_kind": "lease.record", "id_field": "task_id", " kind": "task", "mode": "payload-id"}

Owner: core/authority-model.json; core/contracts.schema.json | Validator: command_capability_rules + command_effect_scope_rules + $defs/GateResult

Lease-bound mutation claim

A lease-bound command is admissible only when authorization proves the exact command capability, holder_authorization independently proves task.execute, both exact Grant claims bind the same subject, Activation and conjunctive effect scope, the referenced Lease is current and ACTIVE at issued_at, and task, generation, fence, candidate, context, and WorkCard digests match exactly.

The command authorization Grant and holder Grant are resolved and validated independently; they may be the same record only when the required command capability is task.execute.

Authorization proof	Source / field	Required capability
command authorization Grant	CommandRequest.authorization.grant_id + grant_claim_digest	resolved command capability
holder Grant	holder_authorization.grant_id	task.execute

Binding equality	Required result
CommandRequest.workcard_task_id == WorkCard.task_id == Lease.task_id	exact match
CommandRequest.subject_id == Lease.holder_subject_id	exact match
CommandRequest.holder_authorization.grant_id == WorkCard.holder_grant_id == Lease.holder_grant_id	exact match
CommandRequest.lease_id == WorkCard.lease_id == Lease.lease_id	exact match
CommandRequest.lease_generation == WorkCard.lease_generation == Lease.generation	exact match
CommandRequest.fencing_token == WorkCard.fencing_token == Lease.fencing_token	exact match
CommandRequest.context_digest == WorkCard.context_digest	exact match
CommandRequest.workcard_digest == sha256(canonical WorkCard)	exact match

Lease-bound commands: artifact.record, run.record, gate.record, finding.record. Required Lease state: ACTIVE. Stale or unresolved: reject.

Owner: core/authority-model.json; core/policy-set.json | Validator: command_mutation_claim_rule + lease_fence + command_capability_scope

Evidence and immutable CAS

Artifact identifies immutable bytes by digest in content-addressed storage. Credit always resolves the exact artifact_id and finalized Artifact-record digest. For artifact_kind=evidence, the event payload carries replay-complete metadata: exact Activation, Candidate, policy, tool, provider, and input digests plus outcome, evidence_purpose, evidence_class, stale, unresolved, and product_credit_eligible. The harness-generated class remains separate from product-execution and receives no product credit.

Evidence field	Rule
digest	immutable CAS content identity
evidence_binding	activation_digest, implementation_closure_digest, candidate_digest, policy_digest, tool_digest, input_digests
outcome	pass, fail, blocked, skipped, error
evidence_class	product-execution, validator, harness-generated, migration
evidence_purpose	must equal the precommitted GateRunDefinition purpose
stale / unresolved	both false for product credit
product_credit_eligible	true only for fresh resolved passing product-execution evidence

Pass, fail, and blocked gate statuses bind exact evidence against an immutable GateRunDefinition precommitted by its owning Task. Skipped requires a reason, carries no Artifact credit, and has pass_credit=false. The definition fixes evidence class and purpose, product-credit requirement, exact target kind, digest and scope, and Candidate, policy, tool, provider, and input digests before a Run or GateResult is submitted. Inline definitions, mixed classes, and payload-digest substitutes reject.

Owner: core/contracts.schema.json; core/semantic-model.json; core/policy-set.json | Validator: $defs/Artifact + $defs/EvidenceBinding + $defs/GateResult + candidate_evidence_binding + credit_requires_evidence

- P-004 lease-fence: Every mutation uses the current active Lease generation and monotonic fencing token.
- P-005 candidate-evidence-binding: Every evidence Artifact requires an explicit diagnostic, gate, or product purpose and binds Activation, candidate, policy, provider/tool, and input digests inside the immutable event payload.
- P-013 safe-export: Export is allowlisted, recursively bounded, symlink-safe, secret-aware, and path-sanitized.
- P-014 closed-acknowledgement: Only a valid CLOSED transition with explicit close acknowledgement releases a mutation slot.

- P-033 lease-only-for-mutation: Mutation WorkCards require the current Lease; read-only WorkCards must not acquire or carry a mutation Lease.
- P-038 canonical-state-machines: Task and Lease transitions are accepted only by the policy-owned state machines; generated lifecycle views cannot invent transitions.
- P-039 decision-target-binding: Every Decision names one typed target and the decision kind, requested scope, candidate, and authority are consistent with that target.
- P-040 credit-requires-evidence: Pass, fail, and blocked gate results cite at least one exact immutable Artifact record reference; skipped results never receive pass credit.

Owner: core/policy-set.json | Validator: lease_fence + candidate_evidence_binding + current_outcome + effective_finding + lease_only_for_mutation + state_machine_transition + decision_target_binding + credit_requires_evidence

Events, replay, and integrity

```
bounded canonical parse
-> compiled Draft 2020-12 schema
-> semantic policy registry
-> Activation integrity
-> capability and conjunctive effect-scope resolution
-> Grant, SOD, Lease, and fence checks
-> candidate and evidence binding
-> bounded atomic event batch
-> rebuildable projection
```

One normalized command with exact authorization, idempotency key, and expected HEAD forms one immutable atomic batch. Under the cross-process writer lock, the runtime refreshes exact HEAD and replay-derived authority state, verifies installed bytes, authorizes and applies the command to a copy, computes the typed state delta, performs file sync, atomic replace, and directory sync, and publishes live state only after the durable append. The batch contains exactly one primary event equal to the validated command payload; event IDs are unique. Commit is O(batch), HEAD lookup O(1), and replay O(events).

EventBatch commitment	Required meaning
previous_authority_commitment	exact previous commitment or the Core-owned genesis value
authority_commitment	commitment over sequence, previous batch, event and state-binding identities
cumulative_event_count	previous count plus current Event count
event_semantic_digest	ordered append-only digest of canonical Events
state_binding_delta	non-empty sorted unique set/delete changes to typed authority leaves
cumulative_state_binding_update_count	previous update count plus current delta length
state_binding_digest	typed-sparse-merkle-v1 post-batch authority root
created_at	UTC with exact second precision

Replay verifies the complete journal prefix. EventStorePolicy is derived live-only from the installed owners with no defaults or nullable critical fields. Checkpoints, SQLite, typed graph state, and sidecars may accelerate rebuild but cannot supply a missing commitment, inject an authority leaf, or change journal meaning.

- P-006 atomic-event-commit: One command produces one immutable batch under lock, compare-head, sync, atomic replace, and recovery.
- P-011 explicit-external-bindings: External standards and providers exist only in explicit digest-bound init records.
- P-015 fail-closed-critical-fields: Missing, unknown, stale, downgraded, or unbound critical fields reject before mutation.
- P-025 one-command-one-batch: Every committed batch binds exactly one subject, validated command digest, command intent digest, authorization digest, and idempotency key.
- P-027 strict-time-order: Timestamps use canonical UTC seconds and issued, heartbeat, expiry, finish, and decision order is semantically checked.

- P-041 command-event-correspondence: One command batch binds the exact authorization claim and contains exactly one primary event equal to the validated command payload; auxiliary events are typed relations only and event IDs are unique.

Owner: core/authority-model.json; core/policy-set.json; core/conformance.json | Validator: EventStorePolicy + EventBatch commitments + atomic_event_commit + historical_revocation + state_command_idempotency + one_command_one_batch + strict_time + command_event_correspondence

Projection, search, and continuation

The SQLite/typed graph is one disposable projection. `next` does not run FTS: it computes a live-only ReadyFrontier from the current acyclic DEPENDS_ON graph, selects the first Task in deterministic frontier order, and returns an immutable read-only WorkCard in NextResult.work_card. `continue` consumes the opaque ReadyFrontier token and returns ContinueResult. Both result envelopes contain exactly six fields and no others: record_type, status, subject_id, activation_digest, work_card, and continuation, in that contract order. status=ready requires a strict Core WorkCard, status=empty requires work_card=null, and continuation is non-null only for a truncated frontier page and contains only its next-page token. A Task is eligible only when every dependency is currently COMPLETED, every required GateResult is current passing credit, and no current OPEN Finding blocks it. WorkCardProjection is internal selected-Task materialization only, is not a public result envelope, and contains no continuation field or token.

For other generic bounded retrieval operations, exact-ID lookup precedes content-aware FTS; RetrievalPage is the separate generic retrieval result and ranking never grants acceptance facts. A broad query selects only a deterministic bounded top-k seed set. When more matches exist, RetrievalPage sets refinement_required=true, returns bounded hints, and exposes no pagination through the unselected corpus. Typed closure is computed only for selected seeds, and their closure union is complete within depth 1-12 and the declared budgets. Silent truncation is forbidden.

A public scheduling continuation is an opaque authenticated ReadyFrontier next-page handle of at most 256 bytes. Its bound state is stored locally, limited to 16 KiB, and binds subject, the current projection.read Grant, scope, Activation, current ReadyFrontier, deterministic order and cursor, HEAD, projection, complete budgets, time bounds, revocation state, and implementation closure. NextResult.continuation and ContinueResult.continuation are null when the frontier page is complete. Separately, RetrievalPage owns generic retrieval continuation and exposes only stream_cursor and next_stream_cursor; removed cursor aliases reject. Its token is not ReadyFrontier scheduling continuation. WorkCardProjection contains no continuation field or token. Continuation overhead remains at most ten percent of the bounded WorkCard. Every page rechecks the Grant; a token is never an authority grant and cannot traverse unselected corpus matches.

WorkCard ceiling	Value
max_bytes	16384
max_entities	32
max_fanout_per_entity	8
max_relations	48
top_k	12

Continuation field	Structural rule
activation_digest	{"$ref": "#/$defs/Digest"}
budgets	{"$ref": "#/$defs/Budget"}
capability_id	{"const": "projection.read"}
cursor	{"minimum": 0, "type": "integer"}
dependency_depth	{"maximum": 12, "minimum": 1, "type": "integer"}
expires_at	{"$ref": "#/$defs/Timestamp"}
grant_claim_digest	{"$ref": "#/$defs/Digest"}
grant_id	{"$ref": "#/$defs/Id"}
head_digest	{"oneOf": [{"$ref": "#/$defs/Digest"}, {"type": "null"}]}
head_event_id	{"oneOf": [{"$ref": "#/$defs/Id"}, {"type": "null"}]}
head_sequence	{"minimum": 0, "type": "integer"}

Continuation field	Structural rule
implementation_closure_digest	{"$ref": "#/$defs/Digest"}
issued_at	{"$ref": "#/$defs/Timestamp"}
normalized_query_digest	{"$ref": "#/$defs/Digest"}
projection_digest	{"$ref": "#/$defs/Digest"}
ranking_id	{"const": "bm25-v1"}
record_type	{"const": "ContinuationTokenClaims"}
requested_scope	{"items": {"$ref": "#/$defs/ScopeSelector"}, "maxItems": 32, "minItems": 1, "type": "array"}
revocation_epoch	{"$ref": "#/$defs/Digest"}
state_id	{"$ref": "#/$defs/Id"}
subject_id	{"$ref": "#/$defs/Id"}
token_version	{"const": 2, "type": "integer"}
traversal_id	{"const": "typed-bfs-v2"}
ttl_seconds	{"maximum": 3600, "minimum": 1, "type": "integer"}

Bounded retrieval rule	Value
broad_query_behavior	deterministic bounded top-k seeds, refinement_required=true, no corpus pagination
continuation_token_bytes_max	256
continuation_state_bytes_max	16384
selected_closure_union_completeness	1
corpus_independent_output_bound	True

- P-007 projection-is-derived: Projection deletion and deterministic replay cannot change normative meaning.
- P-009 bounded-workcard: Every WorkCard stays within both Core hard ceilings and its selected profile byte, entity, relation, fanout, and top-k budgets; every truncation is explicit and losslessly resumable.
- P-024 stale-projection-rejection: Status, next, and WorkCard generation reject a projection whose source head or activation differs.
- P-026 deterministic-retrieval: Ranking ties are deterministic and acceptance-critical facts are added by typed closure, not fuzzy ranking.
- P-032 depth-aware-retrieval-budget: Effective retrieval seed count reserves entity and relation capacity for typed closure at the requested dependency depth; top_k is a ceiling, byte/entity/relation/fanout budgets all remain effective, and no removed or unknown budget field may bypass a ceiling.
- P-042 continuation-authorization-and-completeness: A continuation is authenticated by a random per-Activation secret and binds the exact subject, projection.read Grant claim, capability, requested scope, revocation state, normalized query, deterministic cursor, Activation, HEAD, projection, ranking, complete budgets, depth, implementation closure, issue time, and expiry; its complete page union emits every reachable entity and relation exactly once without loss.

Owner: core/policy-set.json; core/conformance.json; core/contracts.schema.json | Validator: ReadyFrontier + NextResult + ContinueResult + WorkCard + WorkCardProjection + RetrievalPage + projection_derived + bounded_workcard + stale_projection + deterministic_retrieval + depth_aware_retrieval + $defs/ContinuationTokenClaims

Commands and profiles

Surface	Commands
public	init, doctor, status, next, validate, static-admission, continue, audit, refresh, context, skills

Command	Arguments	Grant capability
next	--subject SUBJECT --grant HOLDER_GRANT --query-grant QUERY_GRANT [--depth 1..12]	holder=task.execute; query=projection.read
continue	TOKEN --subject SUBJECT --grant QUERY_GRANT	query=projection.read

The two next Grants are verified independently. Next derives the current ReadyFrontier and returns its first eligible Task as NextResult.work_card; it accepts no FTS expression. Continue requires the explicit subject and current projection.read query Grant and returns ContinueResult for the next frontier page. Both exact envelopes contain only record_type, status, subject_id, activation_digest, work_card, and continuation.

Profile	Tier	Parallel	Default depth	Bytes	Entities	Relations	top_k
baseline	weak-local	1	2	8192	16	24	8
balanced	capable	3	4	12288	24	36	10
extended	strong	6	6	16384	32	48	12

- full-scan-on-init
- provider-discovery-as-authority
- implicit-action-grant
- unbounded-context-expansion
- search-ranking-as-acceptance-proof
- P-052 supported-command-surface: The v1 preset advertises exactly init, doctor, status, next, validate, and continue; no optional or administrative command catalogue may claim an absent executable path.

Owner: presets/semantic-standard.json; core/authority-model.json | Validator: base_user_commands + profiles + forbidden_implicit_behavior + model_tier_rule

Validation, migration, and cutover

The same strict pipeline applies to command, import, replay, rebuild, and export. Empty, unknown, null, stale, degraded, unbound, or cyclic critical records are rejected. A one-time tool outside canonical promin imports supported Tasks, Grants and revocations, Leases, evidence Artifacts, Findings, GateResults, Decisions, and continuation state through the same ingress; generated indexes, reports, and databases are rebuilt. Cutover requires candidate-bound dual-run parity, rollback, deterministic recutover, P0/P1 closure, zero legacy references, and a duly authorized human Decision. Until then cutover=false and deletion=false.

ResearchDraftIntake is bounded transient ingress. Every source binds an immutable Artifact receipt, recomputed content digest, size, media type, provenance, and an active reviewed or source-verified license plan. Claim kinds are research_question, hypothesis, assumption, evidence_claim, and blocked_claim; verification is unverified, source-bound, verified, or blocked. Citation refs identify research sources while evidence refs identify independently resolved Artifact records. Metadata-only sources leave every dependent claim blocked and normative_use_allowed=false.

SanitizedResearchDraft contains only eligible source-bound or independently verified factual candidates. Distribution, marketing, and proof claims remain in an excluded ledger with bounded text and digest, source and evidence refs, and reasons. The result is derived live-only and cannot change GateResult, Decision, acceptance, pass credit, public approval, or distribution state; public_release_approved remains false.

SemanticExport is an export-only semantic-state record, not a package or authority source. It binds exact Candidate, Activation, HEAD, Core, preset, implementation closure, provider receipt, inputs, and actual output Artifact ID and finalized record digest. Output bytes, digest, size, and media type are recomputed from exact CAS bytes; exported records deduplicate by digest and are ordered digest-ascending. Rebuild verifies journal and inventory inputs, performs zero product passes, and atomically publishes a disposable projection.

canonical-name-only, exact-six-core-artifacts, preset-outside-core-digest, exact-one-selected-preset, exact-five-init-records, zero-product-scan-during-init, single-pass-inventory, no-false-success, strict-critical-fields, activation-digest-binding, root-ceiling-without-bootstrap-grant-cycle, grant-scope-expiry-nonce-proof, lease-generation-fence-and-close-ack

candidate-evidence-provider-binding, one-command-one-atomic-batch, deterministic-replay, projection-disposable-and-current, bounded-workcard-through-depth-twelve, typed-reference-domain-range-closure, dependency-graph-acyclic-and-current-ready-frontier, safe-recursive-export, idempotent-state-commands, normalization-collision-rejection, provider-substitution-invalidates-derived-state, rollback-and-recutover-equivalence, zero-forbidden-name-occurrences

zero-unowned-normative-facts, zero-core-provider-lock-in, zero-compiled-cache-files, physical-100k-one-artifact-proxy-per-file, external-standard-release-decision-exact-binding, root-bootstrap-command-limited, deterministic-activation-identity, configuration-exact-idempotency, truthful-operation-reporting, depth-aware-seed-budget, lossless-continuation-in-reference-100k-scenario, read-workcard-without-lease, mutation-workcard-requires-current-lease

command-capability-and-scope-resolution, grant-issuance-chain-closure, team-signatures-cryptographically-verified, required-provider-preflight-before-commit, policy-owned-task-and-lease-state-machines, decision-kind-target-scope-binding, non-skipped-credit-result-has-evidence, exact-command-primary-event-correspondence, conjunctive-scope-containment, authorization-binds-exact-grant-claim, command-effect-target-in-requested-scope, lease-holder-has-task-execute-grant, decision-grant-matches-command-authorization

artifact-kind-resolves-correct-capability, continuation-v2-complete-page-union, candidate-recipe-applied-once, creditable-candidate-immutable-vcs-tree, selected-provider-adapter-evidence, team-signed-cli-end-to-end, current-release-closure-revalidation, finding-bound-resolution-and-waiver-evidence, candidate-delta-within-task-and-grant-scope, verified-head-checkpoint-delta-replay, profile-default-with-core-depth-twelve, canonical-exact-zip-distribution, current-interpreter-and-clean-install-modes

required-no-degradation-tests-fail-closed, continuation-secret-subject-grant-binding, implementation-closure-bound-and-current, verified-inventory-result-provenance, exact-six-base-command-surface, zero-dead-evidence-budget-fields, profile-bound-physical-100k-performance, windows-linux-exact-zip-matrix, fresh-semantic-control-state-per-saturation-iteration, three-zero-new-saturation-iterations, standard-distribution-separate-from-product-acceptance, candidate-evidence-decision-acyclic-chain, evidence-manifest-physical-resolution

evidence-manifest-exact-eight-lane-matrix-resolution, evidence-manifest-four-cp314-supplemental-records, evidence-manifest-path-digest-size-closure, evidence-manifest-nested-and-supplemental-chronology, seven-distinct-role-producer-scopes, signed-standard-decision-authority-proof, semver-from-candidate-binding, typed-human-document-verification, broad-query-bounded-seed-refinement, continuation-token-bytes-at-most-256, inventory-stream-memory-amplification-at-most-32, explicit-scale-marker-selection, authenticated-role-platform-evidence-attestations

installed-platform-observation-closure, handle-relative-evidence-root-safety, raw-scale-summary-recomputation, bounded-incremental-commit-and-compaction, offline-installed-no-degradation, offline-release-online-compatibility, decision-after-evidence-with-bounded-skew, single-derived-platform-closure-owner, truthful-process-invocation-status, product-public-approval-separation, historical-decision-current-eligibility-separation

Owner: core/policy-set.json; core/conformance.json; core/contracts.schema.json | Validator: ResearchDraftIntake + SanitizedResearchDraft + SemanticExport + SemanticStatePayload + ReadyFrontier + required_acceptance + mutation_families + Draft 2020-12 oneOf

Dependency verification

Mode	Execution truth
current-environment	check the interpreter running the command; do not create a nested environment
offline-wheelhouse	create a clean temporary environment; install only from the explicit wheelhouse
online-clean	create a clean temporary environment; install declared dependencies online
none	omit only the dependency smoke and grant no pass credit

No-degradation executes every required test and rejects a skipped required test. It cross-binds the controller platform, CPython version, ABI and tags to the clean-venv executable and SHA-256, the base executable below its base prefix and controller-matching SHA-256, and one SQLite version across controller, validation, and installed observations. One monotonic total deadline covers setup, copies, environment creation, dependency installation, probes, validation, and tests. Every subprocess is contained as a process tree and remaining descendants are terminated after timeout or parent exit. Deadline exhaustion, output overflow, orphaned descendants, or missing progress fails without pass credit.

Platform and Python matrix rows are audit-level compatibility observations only. They are non-authoritative, provide no standalone pass credit, and cannot authorize distribution or product acceptance. Supplemental interpreter rows do not add EvidenceManifest roles.

Each saturation iteration uses a fresh exact five-record, zero-scan control state and never reuses semantic state. The incoming baseline, every iteration control state, and the restored baseline retain byte identities. Audit output contains evidence and logs only; recoverable control state is held in a disjoint workspace-sibling operational-state root. Partial progress is uncredited, and final evidence is published only after three consecutive full zero-new iterations.

Scale mode	Execution truth
python -B -m pytest -q -m "not scale"	focused conformance mode; excludes the physical scale lane
PowerShell: $env:PROMIN_SCALE_WORKSPACE='C:\promin-scale'; python -B -m pytest -q -m scale	Windows physical 100000-file lane; the dedicated workspace is explicit
POSIX: PROMIN_SCALE_WORKSPACE=/tmp/promin-scale python -B -m pytest -q -m scale	Linux physical 100000-file lane; the dedicated workspace is explicit

Owner: tools/promin_validate.py; tools/promin_package.py; tools/promin_no_degradation.py; tools/promin_saturation_audit.py | Validator: dependency-mode dispatch + runtime cross-binding + total deadline + process-tree containment + fresh saturation control state

Technology and provider boundaries

Capability	Task	Forbidden boundary
control-runtime	Parse explicit configuration, canonicalize JSON, and run local CLI orchestration.	{"effect": "reject", "forbids": ["authority-invention", "policy-invention", " provider-binding-invention"], " normative_authority": false}
shape-validation	Validate the single Draft 2020-12 contract bundle with enforced formats and local references.	{"effect": "reject", "forbids": ["authority-decision", "cross-record-semantic-decision"], "normative_authority": false}
content-identity	Compute content digests and content-addressed object identities.	{"effect": "reject", "forbids": ["digest-as-action-authority"], " normative_authority": false}
local-serialization	Provide one local-writer lock, atomic replace, file sync, directory sync, and recovery primitives.	{"effect": "reject", "forbids": ["distributed-consensus-claim"], " normative_authority": false}
query-projection	Build disposable status and bounded typed-retrieval projections; derive ReadyFrontier, materialize one exact WorkCardProjection, and return Core-owned NextResult or ContinueResult envelopes for the six supported CLI commands.	{"effect": "reject", "forbids": ["projection-as-normative-authority"], " normative_authority": false}
filesystem-inventory	Apply the exact candidate recipe in one product pass, use no-follow reads, and publish either a provider-proven immutable VCS tree stream or an explicit non-creditable observation.	{"effect": "reject", "forbids": ["implicit-initialization-scan"], " normative_authority": false}
export-scan	Perform bounded recursive export inspection, allowlisting, secret classification, and symlink rejection.	{"effect": "reject", "forbids": ["package-as-authorization-boundary"], " normative_authority": false}
signature	Dispatch the exact configured signature adapter to sign or verify activation, Grant, and Decision digests for team trust mode, and evidence the invoked adapter identity.	{"effect": "reject", "forbids": ["core-signer-implementation-lock-in"], " normative_authority": false}
build-dependency	Import project-specific build and dependency facts as derived projections.	{"effect": "reject", "forbids": ["semantic-model-redefinition"], " normative_authority": false}

Preset provider class	Capability IDs
required	control-runtime, shape-validation, content-identity, local-serialization, query-projection
additional	filesystem-inventory, export-scan, signature, build-dependency

Capability	Protocol ID	Operations	Receipt
control-runtime	promin.control-runtime.v1	continue, doctor, healthcheck, init, next, provider-binding, status, validate	content-addressed-receipt
shape-validation	promin.shape-validation.v1	command, continue, export, healthcheck, import, provider-binding, rebuild, replay	content-addressed-receipt
content-identity	promin.content-identity.v1	digest-bytes, digest-file, digest-value, healthcheck, provider-binding	content-addressed-receipt
local-serialization	promin.local-serialization.v1	atomic-replace, directory-sync, file-sync, healthcheck, local-writer-lock, provider-binding, recover	content-addressed-receipt
query-projection	promin.query-projection.v1	continue, healthcheck, next, provider-binding, rebuild, status	content-addressed-receipt
filesystem-inventory	promin.filesystem-inventory.v1	healthcheck, immutable-tree-object, immutable-tree-stream, provider-binding	content-addressed-receipt
export-scan	promin.export-scan.v1	bounded-export-scan, healthcheck, provider-binding	content-addressed-receipt
signature	promin.signature.v1	healthcheck, provider-binding, sign-digest, verify-digest	content-addressed-receipt
build-dependency	promin.build-dependency.v1	healthcheck, import-build-facts, provider-binding	content-addressed-receipt

Each operation uses its Core-owned request and response shape and installed content-addressed dependency receipt. ProviderInvocationEvidence binds the exact protocol ID, operation and

operation-contract digest, provider and adapter identity, invocation kind, dependency-receipt digest,and observed outcome. Full output binds exact output_digest, output_size_bytes, andoutput_size_ceiling_bytes; diagnostic stdout and stderr captures are separate, bounded to 1 MiB, andcarry truncation flags. Unknown bindings, reconstructed receipts, and raw provider-specific bypassesreject.

Owner: core/semantic-model.json; presets/semantic-standard.json; core/policy-set.json; core/contracts.schema.json | Validator: technology_capabilities.adapter_protocol + ProviderInvocationEvidence + provider_preflight + provider_substitution

Working examples

```
promin --root <project> init --standard-bundle <promin> --preset <preset> --project-plan
<plans/project.json> --standards-plan <plans/standards.json> --technologies-plan <plans/technologies.json>
--licenses-plan <plans/licenses.json> --authority-plan <plans/authority.json> --emit-plan <canonical-plans>
promin --root <project> init --standard-bundle <promin> --preset <preset> --project-plan
<plans/project.json> --standards-plan <plans/standards.json> --technologies-plan <plans/technologies.json>
--licenses-plan <plans/licenses.json> --authority-plan <plans/authority.json> --review-plan
promin --root <project> init --standard-bundle <promin> --preset <preset> --project-plan
<plans/project.json> --standards-plan <plans/standards.json> --technologies-plan <plans/technologies.json>
--licenses-plan <plans/licenses.json> --authority-plan <plans/authority.json> --dry-run
promin --root <project> init --standard-bundle <promin> --preset <preset> --project-plan
<plans/project.json> --standards-plan <plans/standards.json> --technologies-plan <plans/technologies.json>
--licenses-plan <plans/licenses.json> --authority-plan <plans/authority.json>
promin --root <project> init --standard-bundle <promin> --preset <preset> --project-plan
<plans/project.json> --standards-plan <plans/standards.json> --technologies-plan <plans/technologies.json>
--licenses-plan <plans/licenses.json> --authority-plan <plans/authority.json> --activation-proofs
<proofs.json>

promin --root <project> doctor
promin --root <project> status
promin --root <project> next --subject <id> --grant <holder-grant> --query-grant <query-grant> --depth 2
promin --root <project> validate
promin --root <project> continue <token> --subject <id> --grant <query-grant>
```

The example does not bypass ingress: every state-changing action still passes canonical parse, schema, policies, Activation integrity, capability/scope, Grant/SOD/Lease/fence, evidence binding, and atomic commit. `status`, `next`, and `continue` reject a stale projection or continuation.

`doctor` is the one aggregate live health report for the exact current components core, init, providers, recovery, replay, and projection. Component and rollup statuses are healthy, degraded, failed, or incomplete with precedence failed, incomplete, degraded, healthy. `--no-replay` yields incomplete; a missing or stale projection is never healthy; provider status comes from actual bounded observations. The report always has `report_authoritative=false`, `pass_credit=false`, `product_acceptance_pass=false`, and `product_public_approval=not_approved`.

OperationMetrics is a live-only non-authoritative observation for one command and contains only Core-owned duration, changed-record, event-write, projection-update, runtime-checkpoint, physical-payload-byte, and bytes-per-changed-record fields. DoctorResult and OperationMetrics neither collect, invent, nor allocate token, cost, usage, goal, session, batch, workspace, or per-run usage or per-run accounting.

Owner: presets/semantic-standard.json; core/policy-set.json; core/contracts.schema.json | Validator: base_user_commands + command_capability_scope + critical record definitions

Standard candidate, evidence, and decision

The chain is acyclic: exact candidate ZIP bytes produce StandardReleaseCandidateBinding; exact-package verification and multi-platform closure are recomputed rather than supplied as evidence roles; physically resolved results produce StandardReleaseEvidenceManifest; an active configured authority then signs one explicit approve or reject StandardReleaseDecision; distribution status is derived from that verified chain. VERSION.json describes version and package identity only. Every external JSON record is parsed from one bounded stable byte read. A digest string by itself proves neither evidence nor authority, and the decision cannot change candidate bytes.

StandardReleaseCandidateBinding field	Binding
record_type	exact candidate identity
standard_name	exact candidate identity

StandardReleaseCandidateBinding field	Binding
version	exact candidate identity
archive_sha256	exact candidate identity
archive_bytes	exact candidate identity
archive_member_manifest_digest	exact candidate identity
package_manifest_digest	exact candidate identity
checksums_digest	exact candidate identity
core_bundle_digest	exact candidate identity
preset_digest	exact candidate identity
package_tool_digest	exact candidate identity
validator_digest	exact candidate identity
test_manifest_digest	exact candidate identity
portable_implementation_closure_digest	exact candidate identity
candidate_binding_digest	exact candidate identity
StandardReleaseEvidenceManifest field	**Binding**
record_type	physically resolved evidence set
standard_name	physically resolved evidence set
version	physically resolved evidence set
candidate_binding_digest	physically resolved evidence set
entries	physically resolved evidence set
evidence_manifest_digest	physically resolved evidence set

Required evidence roles: linux, windows, physical-scale, saturation-audit, human-documents, linux-no-degradation, windows-no-degradation

StandardReleaseDecision field	Binding
record_type	signed approve-or-reject claim
decision_id	signed approve-or-reject claim
standard_name	signed approve-or-reject claim
version	signed approve-or-reject claim
candidate_binding_digest	signed approve-or-reject claim
evidence_manifest_digest	signed approve-or-reject claim
outcome	signed approve-or-reject claim
decider_id	signed approve-or-reject claim
release_capability	signed approve-or-reject claim
trust_root_id	signed approve-or-reject claim
signature_provider_id	signed approve-or-reject claim
key_id	signed approve-or-reject claim
nonce	signed approve-or-reject claim
decided_at	signed approve-or-reject claim
signed_claim_digest	signed approve-or-reject claim
signature	signed approve-or-reject claim

Decision verification requires release_capability=standard.distribute, the configured trust root and Ed25519 provider, an active matching key and decider, bounded nonce and decision time, the canonical signed-claim digest, a valid signature, and a separately supplied SHA-256 pin for the exact trust-configuration bytes. A valid signature under an unpinned caller-supplied root is only signature_valid_under_supplied_root and cannot approve. Unknown, revoked, expired, unsigned, replayed-version, unpinned, or mismatched records fail closed. Approve affects only standard distribution; product acceptance remains false, public approval remains not approved, public_release_approved remains false, and cutover and deletion remain false until their independent human decision boundary is satisfied.

Document verification

HumanDocumentVerification is a typed exact-ZIP candidate-bound result. It verifies all four PDFs by exact digest, strict parse, page count, extracted-character count, blank-page and extraction diagnostics, exact regular/bold/italic/mono font digests, and deterministic rebuild. Every page is rendered to a digest-bound PNG, and visual_review_scope covers every rendered page and clipping result; a first-page check or an unspecified visual scope cannot pass. The record always keeps product_acceptance_pass=false.

HumanDocumentVerification field	Structural rule
candidate_binding_digest	$ref="#/$defs/Digest"
deterministic_rebuild	const=true
documents	type="array"; minItems=4; maxItems=4; uniqueItems=true
extraction_diagnostics	type="object"; additionalProperties=false
font_bindings	type="object"; additionalProperties=false
generator_result_digest	$ref="#/$defs/Digest"
invocation	$ref="#/$defs/ReleaseEvidenceInvocation"
page_count	type="integer"; minimum=1
producer	$ref="#/$defs/ReleaseEvidenceProducer"
producer_attestation	$ref="#/$defs/EvidenceProducerAttestation"
product_acceptance_pass	const=false
raw_artifact_manifest_digest	$ref="#/$defs/Digest"
rebuild_digest	$ref="#/$defs/Digest"
record_type	const="HumanDocumentVerification"
render_environment	type="object"; additionalProperties=false
render_manifest	type="array"; minItems=4; maxItems=4096; uniqueItems=true
render_manifest_digest	$ref="#/$defs/Digest"
result_digest	$ref="#/$defs/Digest"
status	const="pass"
visual_review_scope	type="object"; additionalProperties=false

- P-045 current-release-closure: A project-operational release Decision is immutable history; current product release eligibility is derived and becomes false after a blocker, failed gate, stale or missing evidence, implementation drift, or Activation drift. It is separate from the external StandardReleaseDecision for promin distribution.
- P-054 cross-platform-release-evidence: The exact candidate ZIP must pass authenticated Windows and Linux platform observations, clean-install, package, handle-relative path/race, locking, crash, replay, physical scale, offline installed no-degradation, online compatibility, dependency/SBOM/RECORD/license closure, and typed PDF gates. One StandardReleaseEvidenceManifest physically resolves exactly seven unique authority roles, the exact non-authoritative eight-lane CPython 3.13/3.14 online/offline matrix, all four CPython 3.14 supplemental records, and every nested physical record by path, SHA-256, and byte count. All producer attestations bind the same candidate and participate in one replay/SOD/chronology graph; matrix_authoritative, pass_credit, acceptance_pass, product_acceptance_pass, and public approval remain false.
- P-055 standard-release-separation: An external StandardReleaseDecision has explicit approve or reject outcome, binds candidate and evidence manifest digests, follows the manifest-derived maximum evidence completion time, stays within 300 seconds of verifier clock skew, and verifies decider, standard.distribute capability, independently pinned trust root, Ed25519 provider, event-time key validity, nonce, signed claim digest, and signature. The signed decision remains immutable history; current eligibility is derived; product_acceptance_pass is false and product_public_approval is not_approved.
- P-056 required-gates-fail-closed: No-degradation runs the complete required suite only through the clean installed interpreter outside the install source, with isolated mode, cleared PYTHONPATH,

import-origin assertion, zero skipped tests, and full hash-bound JUnit/stdout/stderr bytes.Distribution acceptance executes every required positive, mutation, concurrency, crash, replay,path, package, platform, Python-minor, offline/online, scale, search, PDF, evidence-resolution,chronology, and signature gate; a skipped, missing, stale, blocked, failed, unsigned, unresolved,source-shadowed, or unpinned gate receives no pass credit.

Owner: core/authority-model.json; core/contracts.schema.json; core/conformance.json | Validator: candidate binding + physical evidence resolution + configured signed approve/reject + typed PDF verification